IC-307D: Data Structures & Algorithms Laboratory

Comprehensive Complexity Analysis & Theoretical Performance Reference

1. Foundational Mathematical Definitions

Algorithm analysis measures the growth rate of resource requirements (execution time and physical memory) as input size n approaches infinity. In this laboratory repository, all algorithms are implemented in pure Python from first principles, enabling direct examination of asymptotic bounds without library abstraction.

• **Big-O Notation (Asymptotic Upper Bound, O)**: Formalizes the worst-case ceiling. Formally, $f(n) = O(g(n))$ iff positive constants c and n_0 exist such that $0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$.
• **Big-Omega Notation (Asymptotic Lower Bound, Ω)**: Formalizes the best-case floor. Formally, $f(n) = \Omega(g(n))$ iff positive constants c and n_0 exist such that $0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0$.
• **Big-Theta Notation (Asymptotically Tight Bound, Θ)**: Characterizes algorithms whose upper and lower bounds coincide. Formally, $f(n) = \Theta(g(n))$ iff $c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for constants c_1, c_2.
• **Auxiliary Space Complexity**: Measures exclusively the additional temporary memory allocated during algorithm execution, strictly excluding the input data structure itself. Total space complexity comprises $Space_{total} = Space_{input} + Space_{auxiliary}$.

2. Unit 1: Arrays and Matrices (CO1)

Unit 1 investigates 1D arrays, 2D row-major matrices, 3-tuple sparse matrix compression, and multidimensional matrix calculations. Memory contiguity yields instant $O(1)$ indexed access but imposes $O(n)$ shifting bottlenecks during insertions and deletions.

Program / Operation	Best	Average	Worst	Aux. Space	Theoretical Rationale
1D Insert (at end)	`O(1)`	`O(n)`	`O(n)`	`O(1)`	Appending at tail requires zero shifts; inserting at index 0 forces shifting all n elements rightward.
1D Delete (at end)	`O(1)`	`O(n)`	`O(n)`	`O(1)`	Removing tail is O(1); deleting from position 0 shifts n-1 subsequent items leftward.
1D Linear Search	`O(1)`	`O(n)`	`O(n)`	`O(k)`	Best: target at index 0. Worst: target absent or at index n-1, scanning all n slots.
1D In-Place Rotation	`O(n)`	`O(n)`	`O(n)`	`O(1)`	Reversal algorithm performs 3 subarray reversals: 2(k) + 2(n-k) + 2n = 4n ops = O(n).
2D Cell Access / Update	`O(1)`	`O(1)`	`O(1)`	`O(1)`	Direct coordinate lookup: Addr = Base + (r * cols + c) * ElementSize.
2D 90° Clockwise Rotation	`O(M·N)`	`O(M·N)`	`O(M·N)`	`O(M·N)`	Every cell (r, c) is mapped to (c, M - 1 - r) in an allocated N x M output grid.
Sparse Matrix to Triple	`O(M·N)`	`O(M·N)`	`O(M·N)`	`O(K)`	Full row-major scan of dense grid populating K non-zero triples.
Sparse Transpose	`O(K)`	`O(K log K)`	`O(K log K)`	`O(K)`	Swaps row and column indices for K entries, followed by row-major canonical sorting.
Sparse Addition	`O(1)`	`O(K_A + K_B)`	`O(K_A + K_B)`	`O(K_A + K_B)`	Two-pointer linear merge algorithm across sorted triple streams of matrices A and B.
Matrix Multiplication	`O(R_A·C_A·C_B)`	`O(R_A·C_A·C_B)`	`O(R_A·C_A·C_B)`	`O(R_A·C_B)`	Triple nested loop computing dot products for each cell in resulting R_A x C_B matrix.
Determinant (Laplace)	`O(1)`	`O(n!)`	`O(n!)`	`O(n)`	Laplace cofactor expansion recursively splits n x n matrix into n submatrices of size (n-1).

3. Unit 2: Sorting and Searching Algorithms (CO2)

Unit 2 covers 7 sorting algorithms, binary searching, and hash collision architectures. The comparison-sort decision tree dictates a theoretical lower bound of $\Omega(n \log n)$ worst-case comparisons, which Merge Sort, Quick Sort, and Heap Sort achieve. Radix Sort bypasses this barrier via digit positional distribution in $O(d \cdot (n + b))$ time.

Algorithm	Best	Average	Worst	Aux. Space	Stability / Core Mechanism
Bubble Sort (Optimized)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Stable. Early-exit flag halts after 1 pass on sorted input. Worst: n(n-1)/2 swaps.
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	Unstable. Linear scan for minimum; comparisons invariant to input. Bounds writes to n-1 swaps.
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Stable. Adaptive shifting into sorted prefix; runs in O(n) on pre-sorted or nearly-sorted data.
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Stable. Divide-and-conquer bisection tree depth log n; linear merge requires O(n) aux buffer.
Quick Sort (Median-of-3)	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Unstable. Partitioning around pivot. Median-of-3 avoids worst-case on pre-ordered sequences.
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Unstable. In-place build_max_heap takes O(n); n extractions each take O(log n) sift_down.
Radix Sort (LSD)	$O(d \cdot (n+b))$	$O(d \cdot (n+b))$	$O(d \cdot (n+b))$	$O(n+b)$	Stable. Non-comparison sorting passing d digits using stable Counting Sort (base b=10).
Binary Search (Iterative)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	Halves active search interval [low..high] at each step. Iterative uses constant memory.
Binary Search (Recursive)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Identical comparison count; call-stack incurs O(log n) stack frame memory.
Hash Table (Separate Chaining)	$O(1)$	$O(1)$	$O(n)$	$O(m + n)$	Average O(1) assuming uniform hashing (load factor $\alpha = n/m$). Worst: all keys hash to 1 bucket.
Hash Table (Linear Probing)	$O(1)$	$O(1)$	$O(n)$	$O(m)$	Open addressing. Susceptible to primary clustering under high load factors ($\alpha > 0.7$).

4. Unit 3: Linked Lists (CO1)

Unit 3 evaluates Singly Linked Lists (SLL), Doubly Linked Lists (DLL), Circular Linked Lists (CLL), and their applications (polynomial arithmetic, text buffer, and music playlists). Non-contiguous heap nodes allow $O(1)$ pointer insertions at known locations but forfeit random $O(1)$ indexing, requiring $O(k)$ pointer steps.

Structure / Operation	Best	Average	Worst	Aux. Space	Theoretical Rationale
SLL Insert / Delete at Head	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Direct head pointer update: new_node.next = head; head = new_node. No shifting.
SLL Positional Insert / Delete	$O(1)$	$O(k)$	$O(n)$	$O(1)$	Must traverse k-1 pointer links to reach insertion/deletion site before O(1) pointer rewire.
SLL In-Place Reversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Iterative 3-pointer algorithm (prev, curr, next_node) updates each node's link once.
DLL Insert / Delete at Tail	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Tail pointer enables O(1) operations at the end without traversing from head.
DLL In-Place Reversal	$O(n)$	$O(n)$	$O(n)$	$O(1)$	Iterates across list swapping prev and next pointers for each node, then swaps head and tail.
CLL Traversal / Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Loop terminates when pointer wraps back to head node. Avoids infinite loop via visited check.
Polynomial Addition	$O(1)$	$O(n + m)$	$O(n + m)$	$O(n + m)$	Two-pointer linear merge of sorted descending terms: compares exponents in O(n + m).
Polynomial Multiplication	$O(n \cdot m)$	$O(n \cdot m \cdot K)$	$O(n \cdot m \cdot K)$	$O(n \cdot m)$	Every term in P1 multiplied by each term in P2 (n*m pairs); sorted insertion into result.
Text Editor Line Insertion	$O(1)$	$O(1)$	$O(1)$	$O(1)$	DLL line node insertion at active cursor pointer avoids shifting hundreds of lines.

5. Unit 4: Stacks (CO1)

Unit 4 focuses on LIFO abstract data types across static arrays and dynamic linked nodes, expression conversions (Shunting-Yard algorithm), and undo/redo stacks. All fundamental stack operations (push, pop, peek) operate strictly in *O(1)*.

Operation / Application	Best	Average	Worst	Aux. Space	Theoretical Rationale
Static Stack Push / Pop	O(1)	O(1)	O(1)	O(1)	Index top increment/decrement in preallocated contiguous buffer. Overflow checked in O(1).
Dynamic Stack Push / Pop	O(1)	O(1)	O(1)	O(1)	Heap node prepended/removed at head pointer. Unbounded capacity limited only by RAM.
Infix to Postfix (Shunting-Yard)	O(N)	O(N)	O(N)	O(N)	Each token pushed and popped from operator stack at most once; linear stream processing.
Infix to Prefix Conversion	O(N)	O(N)	O(N)	O(N)	Reverses tokens, swaps parens, applies modified Shunting-Yard, and reverses output: 3*O(N).
Postfix / Prefix Evaluation	O(N)	O(N)	O(N)	O(N)	Single linear pass: operands pushed to stack; operators pop 2 operands and push result.
Undo / Redo Action	O(1)	O(1)	O(1)	O(1)	Popping state from undo_stack and pushing to redo_stack involves two O(1) operations.

6. Unit 5: Queues (CO1)

Unit 5 explores First-In, First-Out (FIFO) mechanics: linear queues, circular arrays solving false overflow, double-ended deques, priority queues (array vs binary heap), and Round Robin process scheduling.

Structure / Algorithm	Best	Average	Worst	Aux. Space	Theoretical Rationale
Static Queue Enqueue / Dequeue	O(1)	O(1)	O(1)	O(1)	Linear array pointers front/rear advance; suffers false overflow once rear hits capacity - 1.
Dynamic Queue Enqueue / Dequeue	O(1)	O(1)	O(1)	O(1)	Singly linked list with dual front & rear pointers; heap allocation per node.
Circular Queue Operations	O(1)	O(1)	O(1)	O(1)	Modulo arithmetic (ptr + 1) % cap reuses vacated slots; count variable tracks full/empty.
Circular Deque Operations	O(1)	O(1)	O(1)	O(1)	Insert/delete at both front and rear in O(1) via bidirectional modular pointer shifts.
Array Priority Queue (Push / Pop)	O(1) / O(n)	O(n)	O(n)	O(1)	Unsorted: push O(1), pop O(n) linear scan. Sorted: push O(n) shift, pop O(1) from tail.
Heap Priority Queue (Push / Pop)	O(1)	O(log n)	O(log n)	O(1)	Binary min-heap enforces heap invariant; sift_up and sift_down traverse height log n.
Round Robin Scheduling	O(N)	O(N·(BT/q))	O(N·(BT/q))	O(N)	Total context switches bounded by sum of (burst_time / quantum) across all N processes.

7. Unit 6: Trees (CO3)

Unit 6 analyzes hierarchical structures: recursive/iterative binary tree traversals, standard Binary Search Trees (BST), self-balancing AVL Trees with 4 rotation primitives, arithmetic Expression Trees, and compiler Symbol Tables.

Structure / Operation	Best	Average	Worst	Aux. Space	Theoretical Rationale
Binary Tree Traversals (All)	O(n)	O(n)	O(n)	O(h)	Every node visited exactly once. Aux space is tree height h (h = log n balanced, h = n skewed).
BST Search / Insert	O(1)	O(log n)	O(n)	O(h)	Follows left/right child based on key comparison. Skews to linear chain O(n) on sorted input.
BST Delete (3 Cases)	O(1)	O(log n)	O(n)	O(h)	Case 1 (leaf): O(1); Case 2 (1 child): O(1); Case 3 (2 children): replace with inorder successor.
AVL Search / Insert / Delete	O(1)	O(log n)	O(log n)	O(log n)	Guaranteed height h ≤ 1.44 log■(n). Rotations rebalance tree in O(1) time after insertion/deletion.

Expression Tree Construction	O(N)	O(N)	O(N)	O(N)	Scans postfix tokens: operands push TreeNode; operators pop 2 subtrees and link as children.
Expression Tree Evaluation	O(N)	O(N)	O(N)	O(h)	Recursive bottom-up postorder evaluation evaluates subtrees before applying operator at root.
BST Symbol Table Lookup	O(1)	O(log n)	O(n)	O(h)	Identifier key lookup retrieves type signatures, scope depths, and source line numbers.

8. Unit 7: Graphs (CO3)

Unit 7 examines graph representations, systematic traversals (BFS & DFS), Dijkstra's single-source shortest path algorithm, and real-world network systems (Social Network, 2D Maze Solver, and DAG Course Prerequisite Scheduler).

Algorithm / Representation	Best	Average	Worst	Aux. Space	Theoretical Rationale
Adjacency Matrix Edge Query	O(1)	O(1)	O(1)	O(V²)	Direct coordinate lookup M[u][v]; requires O(V²) fixed memory regardless of edge count.
Adjacency List Edge Query	O(1)	O(deg(u))	O(V)	O(V + E)	Scans neighbor list of vertex u; optimal space O(V + E) for sparse real-world networks.
Breadth-First Search (BFS)	O(V + E)	O(V + E)	O(V + E)	O(V)	Level-by-level queue exploration visits each vertex and examines each edge once. Finds shortest hops.
Depth-First Search (DFS)	O(V + E)	O(V + E)	O(V + E)	O(V)	Recursive descent explores deeply along paths. Call-stack depth bounded by V in linear chains.
Dijkstra's (Binary Min-Heap)	O((V+E)log V)	O((V+E) log V)	O((V+E) log V)	O(V)	Extract-min performed V times (V log V); edge relaxations push at most E updates (E log V).
Maze Solver (BFS on Grid)	O(1)	O(R · C)	O(R · C)	O(R · C)	Implicit grid graph where |V| = R*C and degree ≤ 4. BFS guarantees shortest path in maze.
Kahn's Topological Sort (DAG)	O(V + E)	O(V + E)	O(V + E)	O(V)	Computes in-degrees, enqueues 0-in-degree vertices, decrements neighbors. Detects cycles if count < V.

9. In-Depth Algorithmic Comparisons

To rigorously synthesize the laboratory findings, this section contrasts competing algorithms and structures across the 7 key technical domains requested.

9.1 Comprehensive Sorting Algorithm Comparison

Algorithm	Best	Average	Worst	Aux. Space	Stable ?	Optimal Domain / Trade-Off
Bubble Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Nearly-sorted data; minimal code footprint (<10 lines).
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)	No	Minimizes physical memory writes (strictly ≤ n - 1 swaps).
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)	Yes	Small sequences (n < 32); adaptive base case in Timsort/Introsort.
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Guaranteed O(n log n) and stability; ideal for linked lists and disks.
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)	No	Fastest general in-place sort due to cache locality and low constants.
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Strict O(1) space guarantee with O(n log n) worst-case ceiling.
Radix Sort (LSD)	O(d·(n+b))	O(d·(n+b))	O(d·(n+b))	O(n+b)	Yes	Fixed-width integers/strings; bypasses comparison lower bound.

9.2 Searching Algorithm Comparison

• **Linear Search**: Requires no preconditions (operates on unsorted arrays or linked lists). Time is *O(n)*, Space is *O(1)*.
• **Binary Search**: Requires pre-sorted contiguous array. Time is *O(log n)*, Space is *O(1)* (iterative) or *O(log n)* (recursive).
• **Separate Chaining Hash Table**: Average lookup is *O(1)*; worst case is *O(n)* under catastrophic collision. Space is *O(m + n)*.
• **Linear Probing Hash Table**: Open addressing eliminates linked bucket pointers. Average lookup is *O(1)*; suffers clustering when load factor $\alpha > 0.7$.

9.3 Contiguous Arrays versus Linked Lists

Feature / Metric	1D Contiguous Array	Singly Linked List	Doubly Linked List
Random Index Access	O(1) direct base address arithmetic	O(n) sequential pointer traversal	O(n) bidirectional traversal (n/2 steps)
Insertion / Deletion at Head	O(n) must shift all n elements	O(1) pointer rewire at head	O(1) pointer rewire at head
Insertion / Deletion at Tail	O(1) amortized append	O(n) without tail pointer; O(1) with tail	O(1) with tail pointer
Arbitrary Positional Insert/Delete	O(n) shifting memory overhead	O(k) traversal + O(1) pointer swap	O(k) traversal + O(1) pointer swap
Memory Overhead per Element	Zero reference overhead (data only)	+8 bytes per node (next pointer)	+16 bytes per node (prev and next)
Hardware Cache Prefetching	Optimal: contiguous L1/L2 cache lines	Poor: fragmented heap memory locations	Poor: fragmented heap memory locations

9.4 Stack and Queue Architecture Trade-offs

• **Static Array Stack/Queue**: Preallocates fixed buffer of size *C*. Incurs zero pointer overhead and provides predictable latency, but risks runtime OverflowError if capacity is exceeded. Linear queues also suffer false overflow when *rear = C - 1*.
• **Dynamic Linked Stack/Queue**: Allocates nodes on demand on heap. Never overflows unless system RAM is exhausted, but consumes extra pointer references per item (8 bytes per node).
• **Circular Queue**: Employs modulo arithmetic *(ptr + 1) % capacity* to wrap pointers around the physical array boundary, completely eliminating false overflow and maintaining strict *O(1)* latency.
• **Priority Queue (Array vs Heap)**: Unsorted array push is *O(1)* but pop is *O(n)*. Sorted array push is *O(n)* but pop is *O(1)*. Binary min-heap provides an optimal compromise: push is *O(log n)* and pop is *O(log n)*.

9.5 Binary Search Tree (BST) versus Self-Balancing AVL Tree

Metric / Operational Invariant	Standard Binary Search Tree (BST)	Self-Balancing AVL Tree
Height Guarantee	Unbounded: h = n in worst case (linear chain)	Strictly bounded: $h \leq 1.44 \cdot \log_2(n)$
Search Time (Average / Worst)	Average: O(log n) | Worst: O(n)	Average: O(log n) | Worst: O(log n) Guaranteed
Insertion Time (Average / Worst)	Average: O(log n) | Worst: O(n)	Average: O(log n) | Worst: O(log n) Guaranteed
Deletion Time (Average / Worst)	Average: O(log n) | Worst: O(n)	Average: O(log n) | Worst: O(log n) Guaranteed
Rebalancing Overhead	None (tree remains skewed on sequential input)	O(1) rotations (LL, RR, LR, RL) triggered when |BF| > 1
Auxiliary Node Metadata	None (key, left, right)	Stores node height (or balance factor) per node
Best Production Use Case	Read-heavy workloads with known random inputs	Lookup-intensive databases demanding strict latency ceilings

9.6 Graph Traversal Comparison: BFS versus DFS

• **Breadth-First Search (BFS)**: Traverses graph level-by-level using a FIFO queue. Time: *O(V + E)*. Auxiliary Space: *O(V)* (queue holds entire level width). Guarantees shortest path in unweighted graphs because vertices are visited in non-decreasing order of edge hops.
• **Depth-First Search (DFS)**: Traverses deeply along each branch using recursion or an explicit LIFO stack before backtracking. Time: *O(V + E)*. Auxiliary Space: *O(V)* (call-stack depth bounded by graph diameter). Optimal for topological sorting, cycle detection, and maze exploration, but does not guarantee shortest paths.

9.7 Shortest Path Strategies: Unweighted BFS versus Dijkstra's Algorithm

• **Unweighted BFS**: Assumes all edge weights are identical ($w(u, v) = 1$). Operates in $O(V + E)$ time using a basic FIFO queue. Cannot handle non-uniform edge weights because level order no longer reflects accumulated path cost.
• **Dijkstra's Algorithm**: Solves Single-Source Shortest Paths on weighted graphs with non-negative edge weights ($w(u, v) \geq 0$). Using a custom binary min-heap, it achieves $O((V + E) \log V)$ time and $O(V)$ auxiliary space. Maintains the **relaxation invariant**: $dist[v] = \min(dist[v], dist[u] + w(u, v))$.
• **Negative Weight Limitation**: Dijkstra's greedy invariant assumes that once a vertex is popped from the min-heap, its shortest distance is finalized. Negative edge weights violate this monotonicity, necessitating the Bellman-Ford algorithm ($O(V \cdot E)$).

10. Theoretical Complexity versus Empirical Hardware Realities

While asymptotic Big-O complexity accurately predicts algorithm scalability as $n \to \infty$, wall-clock execution time on physical x86_64 architectures is heavily governed by hardware and implementation constants:
1. **Cache Locality**: Contiguous array algorithms (like Quick Sort and Binary Search) exploit CPU L1/L2 data cache lines (64 bytes), prefetching adjacent elements into cache. In contrast, linked list and tree nodes reside at scattered heap addresses, inducing frequent CPU cache misses.
2. **Constant Factors**: Merge Sort executes fewer comparisons than Quick Sort, but Quick Sort consistently runs faster in practice because it operates completely in-place without the memory allocation and copying overhead of Merge Sort's auxiliary buffer.
3. **Interpreter Overhead**: Pure Python code incurs interpreter dispatch overhead per instruction. The algorithms implemented in this repository rigorously maintain authentic data structure invariants from first principles, demonstrating predictable big-O scaling across all 7 units.